# SHARPER: Towards an Automated Verified Repair Tool for C#

Jyoti Prakash[1], Manish Raj Moriche[2], Sahal Abdullah[1], Dimistrios Dafnis[1], Adam Alami[2], and Abhishek Tiwari[1]

[1] SDU Software Engineering, University of Southern Denmark, Odense, Denmark
jyp@mmmi.sdu.dk sahab@mmmi.sdu.dk
didaf25@student.sdu.dk abti@mmmi.sdu.dk
[2] Center for Industrial Software, University of Southern Denmark, Sønderborg, Denmark
maraj24@student.sdu.dk adal@mmmi.sdu.dk

**Abstract.** We present SHARPER, an automated repair tool for the C# applications. SHARPER requires an failing input, such as a failing test case, and records the execution trace to infer the constraint that is violated at the crash site. The violation is the condition expressed in predicate logic that, if satisfied, would have resulted in normal execution without failures. Based on the violation, SHARPER introduces a fix, and then verifies it by propagating the condition in static analyser. If the fix is found to be correct, then it patches the code with the verified fix, otherwise, the fix is discarded.

**Keywords:** Program repair · Patch correctness, C#

## 1 Introduction

The increasing effectiveness of the code-generating systems has changed the bottleneck in software development, and more specifically, in automated program repair. Previously, generating the correct patches relied on a fixed grammar (usually confined to linear-integer arithmetic) and a set of examples. The patches that came from this process was verified, however, often these were confined to simple arithmetic expression, which is a far cry from what is used in practice. The evolution of large language models (LLMs) has changed the discourse, where the focus has shifted from the generating the precise candidate patches to verifying whether the candidates patches are correct. This distinction is particularly important for automated program repair when a patch that compiles and passes an existing test suite is not necessarily semantically correct. Tests provide only a finite approximation of intended behaviour and may permit patches that over-fit the observed failures. Consequently, practical deployment of automated repair in critical software requires mechanisms that provide stronger assurance about candidate patches.

We investigate this problem for C# programs by combining automated repair with established techniques from static analysis and formal verification. Rather than treating patch generation and validation as independent stages, our approach uses program-analysis information to guide repair and verification techniques to establish whether candidate modifications satisfy the required behavioural properties. This enables the repair process to reject patches that merely eliminate an observed failure while failing to preserve the intended semantics of the program.

Our objective is therefore not merely to generate plausible patches, but to generate patches for which correctness can be justified with respect to an explicit specification. We present a prototype implementation of this approach for C#and evaluate it on an industrial codebase. In the studied case, the prototype identified and repaired the targeted defect and produced a modification structurally comparable to the patch developed independently by a human engineer. An evaluation provides preliminary evidence that verification-guided repair can be integrated into realistic C#development workflows. This goal builds on generate-and-validate automated program repair, surveyed by Monperrus [5] and exemplified by GenProg [3].

## 2 Tool Overview

SHARPER consists of three stages: (1) Record-and-run, which records the runtime trace, (2) constraint generator, that generates the constraint, and a (3) patch synthesizer that synthesizes the patch based on the constraint and validates it against the test-suite. SHARPER is implemented on the top of Roslyn and integrated with the .NET build and runtime environment. Currently, SHARPER supports C#with the infrastructure to extend it various other languages from .NET.

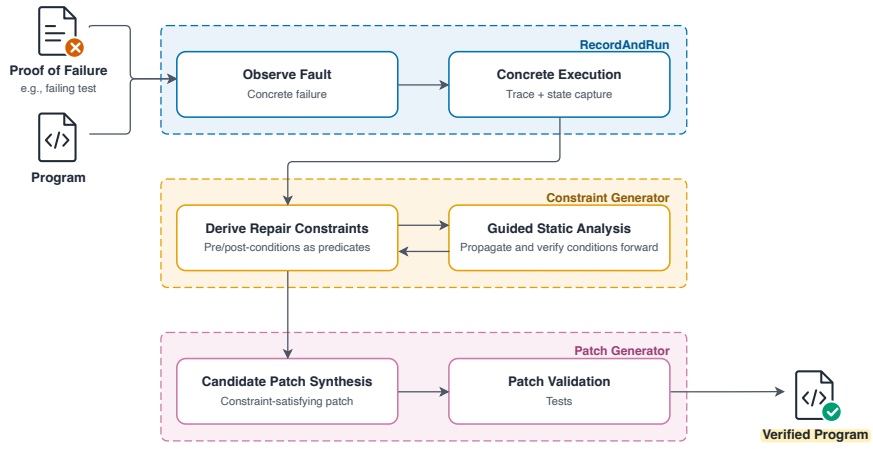



**Fig. 1.** Architecture of SHARPER

### 2.1 Record and Run

It rewrites $P$ as $P'$ to record events during execution on the reproducer $r$. In the process, it records the function entry and exit, the state of the program at each statement, and the branch taken for loop and conditions. It also implements some optimisations to reduce the footprint for storing the program state. One of such footprint, is to only store the relevant values rather than tracing the whole program. We represent the resulting dynamic trace as the finite sequence $\tau_{P,r} = \langle e_1, \ldots, e_n \rangle$ where $e_i = (k_i, \ell_i, d_i), and\ k_i$ is an event kind, $\ell_i$ is its program location, and $d_i$ contains the observations recorded for that occurrence. Event kinds include method entry, branch and loop outcomes, assignments, array indices, checked arithmetic operations, runtime types, and failure. The sequence is ordered by execution, i.e., repeated visits to a location are distinct events. For a failing run, the terminal failure event identifies $\ell_f$, the failure location. Thus $\tau_{P,r}$ describes one observed execution, not all feasible executions of $P$.

### 2.2 Constraint Generator

Given a failure location $\ell_f$, the analysis computes a backward slice $S(\ell_f)$ containing the program locations on which the failure is data-dependent or control-dependent, following classical program slicing [9]. At $\ell_f$, an exception-specific template is instantiated from the observed program state to obtain an initial failure predicate $\Phi_f$. The current implementation provides templates for array-bounds violations, null dereferences, and invalid casts, with limited support for selected C# collection types. The initial predicate $\Phi_f$ depends on the runtime failure. Table 1 summarizes the templates currently supported for array-bounds violations, null dereferences, and invalid casts.

**Table 1.** Failure predicates used to initialize backward propagation.

| Failure kind | Predicate |
|---|---|
| Array bounds | $\Phi_f^{\mathsf{bounds}}(a, i) \triangleq (i < 0) \vee (i \geq \mathsf{len}(a))$ |
| Null dereference | $\Phi_f^{\mathsf{null}}(v) \triangleq (v = \mathsf{null})$ |
| Invalid cast | $\Phi_f^{\mathsf{cast}}(v, \tau) \triangleq (v \neq \mathsf{null}) \wedge \neg\mathsf{compatible}(v, \tau)$ |

Let $\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \Phi \rangle$ denote the backward propagation of predicate $\Phi'$ from a location $\ell'$ to its predecessor $\ell$, resulting in predicate $\Phi$. Figure 2 summarizes the inference rules for this relation and for identifying root-cause candidates.

The INITIALIZE rule in Figure 2 applies the selected template to the observed failure state $\sigma_f$ at $\ell_f$. When traversing an assignment $x := e$ backwards, the analysis substitutes $e$ for each free occurrence of $x$ in the propagated predicate, as specified by ASSIGN in Figure 2. Here, $\Phi'[e/x]$ denotes capture-avoiding substitution, and $\mathsf{fv}(\Phi')$ is the set of free variables of $\Phi'$. If $x$ does not occur in the predicate, it remains unchanged, as specified by ASSIGN-IRREL. If the execution

$$\frac{\ell \xrightarrow{x:=e} \ell' \qquad x \in \mathsf{fv}(\Phi')}{\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \Phi'[e/x] \rangle}\ \textsc{Assign} \qquad \frac{\ell \xrightarrow{x:=e} \ell' \qquad x \notin \mathsf{fv}(\Phi')}{\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \Phi' \rangle}\ \textsc{Assign-Irrel}$$

$$\frac{\ell \xrightarrow{\rho} \ell'}{\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \rho \wedge \Phi' \rangle}\ \textsc{Guard-T} \qquad \frac{\ell \xrightarrow{\neg\rho} \ell'}{\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \neg\rho \wedge \Phi' \rangle}\ \textsc{Guard-F}$$

$$\frac{\ell \xrightarrow{s} \ell' \qquad \mathsf{def}(s) \cap \mathsf{fv}(\Phi') = \varnothing \qquad \mathsf{guard}(s) = \varnothing}{\langle \ell', \Phi' \rangle \Longrightarrow \langle \ell, \Phi' \rangle}\ \textsc{Frame}$$

$$\frac{\mathsf{succ}_S(\ell) = \{\ell_1, \ldots, \ell_n\} \qquad \langle \ell_i, \Phi_i \rangle \Longrightarrow \langle \ell, \Psi_i \rangle \quad \text{for every } i \in \{1, \ldots, n\}}{\langle \{\ell_1, \ldots, \ell_n\}, \{\Phi_1, \ldots, \Phi_n\} \rangle \Longrightarrow \left\langle \ell, \bigvee_{i=1}^{n} \Psi_i \right\rangle}\ \textsc{Join}$$

$$\frac{\mathsf{template}(\ell_f, \sigma_f) = \Phi_f}{\mathsf{init}(\ell_f, \sigma_f) \Longrightarrow \langle \ell_f, \Phi_f \rangle}\ \textsc{Initialize}$$

$$\frac{\ell \in S(\ell_f) \qquad \sigma_\ell \xrightarrow{s_\ell} \sigma_{\ell'} \qquad \sigma_\ell \not\models \Phi_\ell \qquad \sigma_{\ell'} \models \Phi_{\ell'}}{\ell \in R(\ell_f)}\ \textsc{Root}$$

**Fig. 2.** Backward predicate propagation and root-cause inference rules.

reaches $\ell'$ only when a branch condition $\rho$ is true, the analysis conjoins $\rho$ with the propagated predicate, as specified by Guard-T. On the alternative branch it conjoins $\neg\rho$, as specified by Guard-F.

A statement that neither modifies a variable in the propagated predicate nor contributes a relevant path condition leaves the predicate unchanged, as specified by Frame in Figure 2. Here, $\mathsf{def}(s)$ denotes the variables defined by $s$, and $\mathsf{guard}(s)$ denotes any path condition contributed by $s$. At a join with multiple relevant successors, the analysis combines the propagated predicates disjunctively. Figure 2 formalizes this with Join; $\mathsf{succ}_S(\ell)$ denotes the successors of $\ell$ in the backward slice. The disjunction accounts for failure along any of those successor paths.

Let $\sigma_\ell$ denote the state observed at location $\ell$ during the failing execution, and let $\Phi_\ell$ be the predicate propagated backwards from $\ell_f$ to $\ell$. A location is a root-cause candidate when the failure-enabling predicate changes from false to true across its statement, as captured by Root in Figure 2. Accordingly, the candidate set is

$$R(\ell_f) \triangleq \left\{ \ell \in S(\ell_f) \;\middle|\; \begin{array}{l} \sigma_\ell \xrightarrow{s_\ell} \sigma_{\ell'}, \\ \sigma_\ell \not\models \Phi_\ell, \text{ and} \\ \sigma_{\ell'} \models \Phi_{\ell'} \end{array} \right\}.$$

For each location $\ell'_f$ a transition boundary at which a condition necessary for the observed failure is established. These locations are passed to repair generation, which instantiates repair templates intended to prevent the failure predicate from being established along the failing execution.

To assess a generated repair condition $\Phi(v)$, Sharper assumes that the condition holds and propagates it forward through the control-flow graph. This

analysis checks whether enforcing the condition introduces a violation at a later program location. If a violation is found, SHARPER identifies its root cause, derives an additional repair condition, and repeats the analysis. Forward propagation uses rules analogous to the backward rules in Figure 2 as we skip it here owing to page limits.

```
using System;
class Program
{
  static void ProcessBuffers(int check, int i) {
    int[] buf1 = new int[5]; int[] buf2 = new int[2];

    while (i < check) {
      buf1[i] = i + 1;
      ++i;
    }
    // ....
    while (i >= 0) {
      buf2[i] = i + 1;
      --i;
    }
  }
}
```

**Listing 1.1.** C#buffer-access reproducer.

### 2.3 Patch Generator

For each root-cause candidate, the patch generator selects a repair template compatible with the propagated predicate and the statement at that location. The predicate determines the guard needed to prevent the failure and it combines with the template to generate the repair candidate. The repair candidate is patched and validated against the test-suite and the reproducer.

### 2.4 Running Example

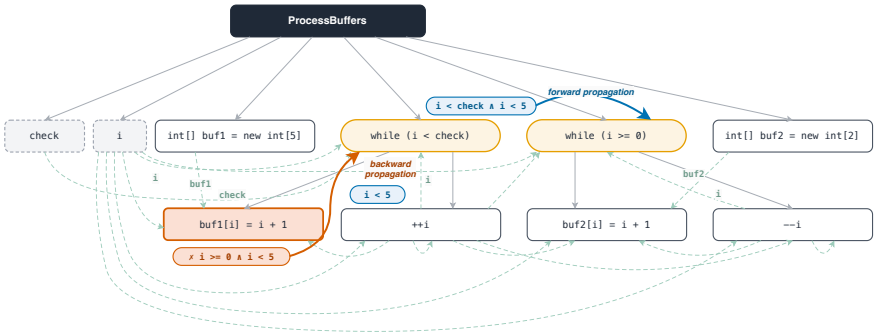



**Fig. 3.** Backward derivation and forward validation of a repair condition in `ProcessBuffers`.

Figure 3 instantiates the analysis on `ProcessBuffers` from Listing 1.1; assume the initial index is $i = 0$ and $check > 5$. The first access writes to `buf1`, whose length is 5. Its array-bounds failure predicate from Table 1 is $i < 0 \vee i \geq 5$, so the corresponding safety condition is $0 \leq i < 5$. Backward propagation through the increment and the first loop derives the candidate guard $i < check \wedge i < 5$, shown in blue; under the stated initial-state assumption, this prevents that loop from accessing `buf1[5]`. The dashed edges show the data and control dependencies used in deriving the condition. The blue forward arrow then checks this candidate against the remainder of the execution: the first loop exits with $i = 5$, but the second loop's guard $i \geq 0$ still permits an access to `buf2[5]`, although `buf2` has length 2. Thus the first guard addresses the failure at `buf1[i]` but is not a whole-program repair. Forward validation exposes the later violation and motivates deriving and checking a repair condition for the second access as well.

## 3 Related Work

Generate-and-validate automated program repair has been studied extensively [5]; GenProg is a representative system that searches for patches against a test suite [3]. Such test-plausible patches can overfit the available tests [8,6], motivating patch-correctness assessment techniques based on generated tests and other analyses [11,10]. In contrast to semantic repair systems such as Angelix and concolic program repair, which use symbolic reasoning to synthesize patches [4,7], SHARPER is independent of the patch generator and validates candidate patches against a failure condition across the patched program. Its failure localization combines static program slicing [9] with trace-guided dynamic slicing [2,1], and reports surviving paths as witnesses rather than returning only a plausibility score.

## 4 Conclusion

SHARPER is an open-source oracle that validates candidate patches for `C#`beyond the bug reproducer. It turns one failing execution into a failure condition by slicing the control-flow graph, and checks every patched program statically for any remaining path to the failure. Planned work includes deriving failure conditions from several reproducers, feeding witnesses back to the patch synthesiser, and checking that patches preserve the behaviour of passing tests.

### Data-Availability Statement.

The artifacts are available at: [TODO: insert link ]

## References

1. Agrawal, H., Horgan, J.R.: Dynamic program slicing. In: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation (PLDI). pp. 246–256. ACM (1990). `https://doi.org/10.1145/93542.93576`

2. Korel, B., Laski, J.: Dynamic program slicing. Information Processing Letters **29**(3), 155–163 (1988). https://doi.org/10.1016/0020-0190(88)90054-3
3. Le Goues, C., Nguyen, T., Forrest, S., Weimer, W.: GenProg: A generic method for automatic software repair. IEEE Transactions on Software Engineering **38**(1), 54–72 (2012). https://doi.org/10.1109/TSE.2011.104
4. Mechtaev, S., Yi, J., Roychoudhury, A.: Angelix: Scalable multiline program patch synthesis via symbolic analysis. In: Proceedings of the 38th International Conference on Software Engineering (ICSE). pp. 691–701. ACM (2016). https://doi.org/10.1145/2884781.2884807
5. Monperrus, M.: Automatic software repair: A bibliography. ACM Computing Surveys **51**(1), 17:1–17:24 (2018). https://doi.org/10.1145/3105906
6. Qi, Z., Long, F., Achour, S., Rinard, M.: An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In: Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA). pp. 24–36. ACM (2015). https://doi.org/10.1145/2771783.2771791
7. Shariffdeen, R., Noller, Y., Grunske, L., Roychoudhury, A.: Concolic program repair. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI). pp. 390–405. ACM (2021). https://doi.org/10.1145/3453483.3454051
8. Smith, E.K., Barr, E.T., Le Goues, C., Brun, Y.: Is the cure worse than the disease? Overfitting in automated program repair. In: Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE). pp. 532–543. ACM (2015). https://doi.org/10.1145/2786805.2786825
9. Weiser, M.: Program slicing. IEEE Transactions on Software Engineering **SE-10**(4), 352–357 (1984). https://doi.org/10.1109/TSE.1984.5010248
10. Xin, Q., Reiss, S.P.: Identifying test-suite-overfitted patches through test case generation. In: Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). pp. 226–236. ACM (2017). https://doi.org/10.1145/3092703.3092718
11. Xiong, Y., Liu, X., Zeng, M., Zhang, L., Huang, G.: Identifying patch correctness in test-based program repair. In: Proceedings of the 40th International Conference on Software Engineering (ICSE). pp. 789–799. ACM (2018). https://doi.org/10.1145/3180155.3180182